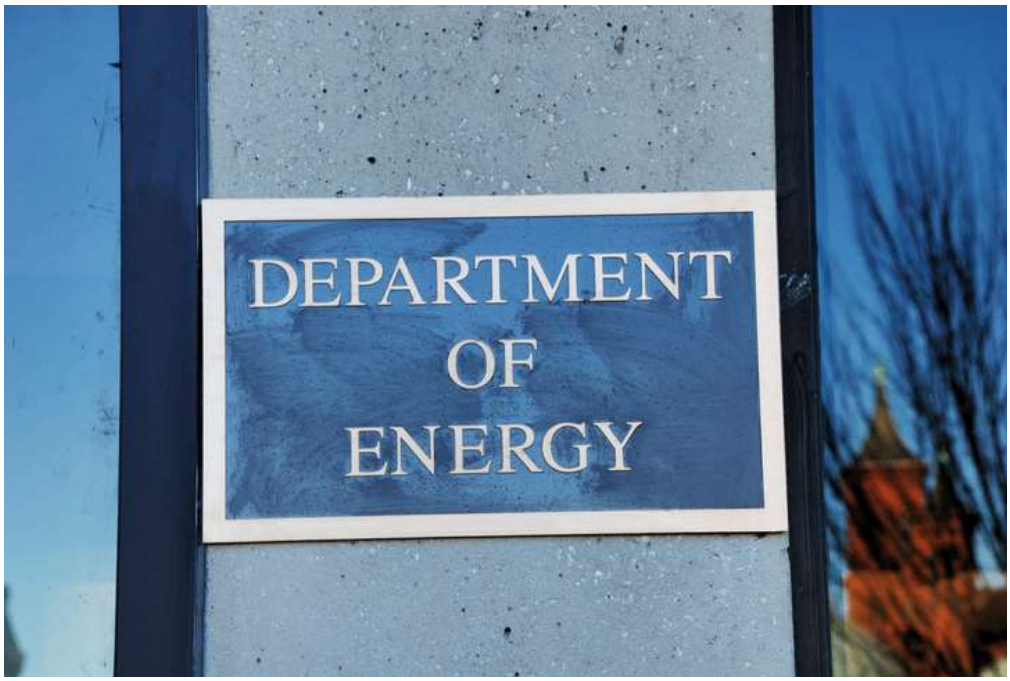

You've recently started a new position as a Data Engineer at an energy company. Previously, analysts on other teams had to manually retrieve and clean data every quarter to understand changes in the sales and capability of different energy types. This process normally took days and was something that most analytsts dreaded. Your job is to automate this process by building a data pipeline. You'll write this data pipeline to pull data each month, helping to provide more rapid insights and free up time for your data consumers.

You will achieve this using the `pandas` library and its powerful parsing features. You'll be working with two raw files; `electricity_sales.csv` and `electricity_capability_nested.json`.

Below, you'll find a data dictionary for the `electricity_sales.csv` dataset, which you'll be transforming in just a bit. Good luck!

Field	Data Type
period	`str`
stateid	`str`
stateDescription	`str`
sectorid	`str`
sectorName	`str`
price	`float`
price-units	`str`

```python
import pandas as pd
import json
```

```python
def extract_tabular_data(file_path: str):
    """Extract data from a tabular file_format, with pandas."""
    if file_path.endswith(".csv"):
        return pd.read_csv(file_path)

    elif file_path.endswith(".parquet"):
        return pd.read_parquet(file_path)

    else:
        raise Exception("Warning: Invalid file extension. Please try with .csv or
.parquet!")
```

```python
def extract_json_data(file_path):
    """Extract and flatten data from a JSON file."""
    # First, read in the JSON file into memory using the json library
    with open(file_path, "r") as json_file:
        raw_data = json.load(json_file)


    return pd.json_normalize(raw_data)
```

```python
def transform_electricity_sales_data(raw_data: pd.DataFrame):
    """
    Transform electricity sales to find the total amount of electricity sold
    in the residential and transportation sectors.

    To transform the electricity sales data, you'll need to do the following:
    - Drop any records with NA values in the `price` column. Do this inplace.
    - Only keep records with a `sectorName` of "residential" or "transportation".
    - Create a `month` column using the first 4 characters of the values in
`period`.
    - Create a `year` column using the last 2 characters of the values in `period`.
    - Return the transformed `DataFrame`, keeping only the columns `year`, `month`,
`stateid`, `price` and `price-units`.
    """
    # Drop any records with a null value
    raw_data.dropna(subset=["price"], inplace=True)

    # Only keep residential and transformation records
    cleaned_df = raw_data.loc[raw_data["sectorName"].isin(["residential",
"transportation"]), :]

    # Create year and month columns
    cleaned_df["year"] = cleaned_df["period"].str[0:4]
    cleaned_df["month"] = cleaned_df["period"].str[5:]

    # Only keep columns period, stateId, sector, value, units
    cleaned_df = cleaned_df.loc[:, ["year", "month", "stateid", "price", "price-
units"]]

    return cleaned_df
```

```python
def load(dataframe: pd.DataFrame, file_path: str):
    """Load a DataFrame to a file in either CSV or Parquet format."""
    # Check to see if the file path ends with .csv or .parquet
    if file_path.endswith(".csv"):
        dataframe.to_csv(file_path)

    elif file_path.endswith(".parquet"):
        dataframe.to_parquet(file_path)

    # Otherwise, throw an exception
    else: raise Exception(f"Warning: {file_path} is not a valid file type. Please
try again!")
```

```
# Ready for the moment of truth? It's time to test the functions that you wrote!
raw_electricity_capability_df =
extract_json_data("electricity_capability_nested.json")
raw_electricity_sales_df = extract_tabular_data("electricity_sales.csv")

cleaned_electricity_sales_df =
transform_electricity_sales_data(raw_electricity_sales_df)

load(raw_electricity_capability_df, "loaded__electricity_capability.parquet")
load(cleaned_electricity_sales_df, "loaded__electricity_sales.csv")
```